Transcript for Presentation on Difference Equations

Your Name

October 16, 2024

Slide 1: Overview of Difference Equations

Welcome! In this presentation, we'll be exploring the concept of **difference equations**.

Difference equations are widely used for **modeling dynamic systems** by tracking the relationship between values at different time steps.

Next, let's discuss some **key examples and applications**. These include **control systems**, **signal processing**, and **finance**.

Each of these fields relies on the precise modeling of discrete time steps.

Finally, difference equations share a close **relationship with differential equations**, which describe continuous-time systems. In many cases, difference equations provide a discrete approximation.

Slide 1a: Importance of Difference Equations

Why are difference equations so important?

First, they're essential for **modeling dynamic systems**. They capture changes between discrete time steps—ideal for systems like population growth or economic trends.

Next, they serve as a **bridge to differential equations**. When data is sampled at discrete intervals—such as digital signals—difference equations offer a way to model those systems effectively.

Slide 1b: Applications of Difference Equations

Here are some key **application areas** of difference equations.

In **control systems**, they're used in feedback loops, system stability, and controller design.

In **signal processing**, difference equations help with filtering and analyzing discrete-time signals.

In **finance**, they model investments, loans, and asset dynamics over time.

And finally, in **epidemiology**, they predict the spread of diseases using discrete time models.

Slide 2: Definitions (Scalar and Vector Case)

Let's look at the **definition of difference equations**.

A **scalar difference equation** relates successive values of a scalar sequence. Here's an example:

$$y(n+1) = a(1)y(n) + a(2)y(n-1) + \cdots + a(k)y(n-k+1) + b.$$

When the output becomes a **vector**, we use a **vector difference equation**:

$$y_i(n+1) = \sum_{j=1}^{k} a_{ij} y_j(n) + b_i.$$

Scalar equations are a special case of vector equations when the output has only one dimension.

Slide 2b: Higher-Order vs. First-Order Difference Equations

Higher-order scalar equations involve **multiple past values** of the sequence, such as $y(n)$ and $y(n-1)$.

However, these can be converted into **first-order vector equations** involving only the current and next time steps. For example:

$$x(n+1) = Ax(n) + Bu(n).$$

Slide 2c and 2d: Converting Higher-Order Scalar to First-Order Vector Equations

Let's see how to convert higher-order scalar equations into first-order vector equations.

Start with the scalar equation:

$$y(n+1) = a(1)y(n) + a(2)y(n-1) + b.$$

We introduce **state variables** to capture past steps:

$$x_1(n) = y(n), \quad x_2(n) = y(n-1).$$

Now, we can express the system using **state-space equations**:

$$x_1(n+1) = a(1)x_1(n) + a(2)x_2(n) + b, \quad x_2(n+1) = x_1(n).$$

Slide 3a: State Variables and Outputs

In state-space models, the **state vector** $x(n)$ represents the internal state at time n. Its dimension is d, the number of states.

The **output vector** $y(n)$ provides the observed values. Its dimension is m, the number of outputs.

Slide 3b: State-Space Output Equation

The output $y(n)$ is related to the state through the matrix C:

$$y(n) = Cx(n).$$

Here, C is an $m \times d$ matrix, where m is the number of outputs and d is the number of states.

Slide 3c: Control Inputs, Noise, and State Transition

Control inputs $u(n)$ represent external influences on the system. These inputs have dimension p.

Noise inputs $w(n)$ account for uncertainties in the system, with dimension q.

The state transition equation is:

$$x(n+1) = Ax(n) + Bu(n) + Gw(n).$$

Slide 4a: Defining the State for a Simple System

Let's consider a simple system described by:

$$y(n+1) = ay(n) + bu(n).$$

We define the **state variable** $x(n) = y(n)$. This makes the state and output identical, though in more general cases, they may differ.

Slide 4b: State-Space Representation (General Form)

The general state-space form is:

$$x(n+1) = Ax(n) + Bu(n).$$

The output is:

$$y(n) = Cx(n).$$

Here, A, B, and C are matrices of appropriate dimensions.

Slide 5a1: AR Component of ARMA Models

The AR component of an ARMA model is:

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n).$$

Here, μ_{AR} is a constant drift term, $a(i)$ are the AR coefficients, and $w(n)$ is white noise.

Slide 5a2: MA Component and Combined ARMA Model

The MA component is:

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j).$$

The full ARMA model is:

$$y(n) = \mu + \sum_{i=1}^{p} a(i)y(n-i) + w(n) + \sum_{j=1}^{q} b(j)w(n-j).$$

Slide 5b and 5c: Explanation of Mu in ARMA Models

In the AR component, μ_{AR} shifts the series level.

In the MA component, μ_{MA} adjusts the long-term mean. These terms combine into $\mu = \mu_{AR} + \mu_{MA}$.